

JavaScript

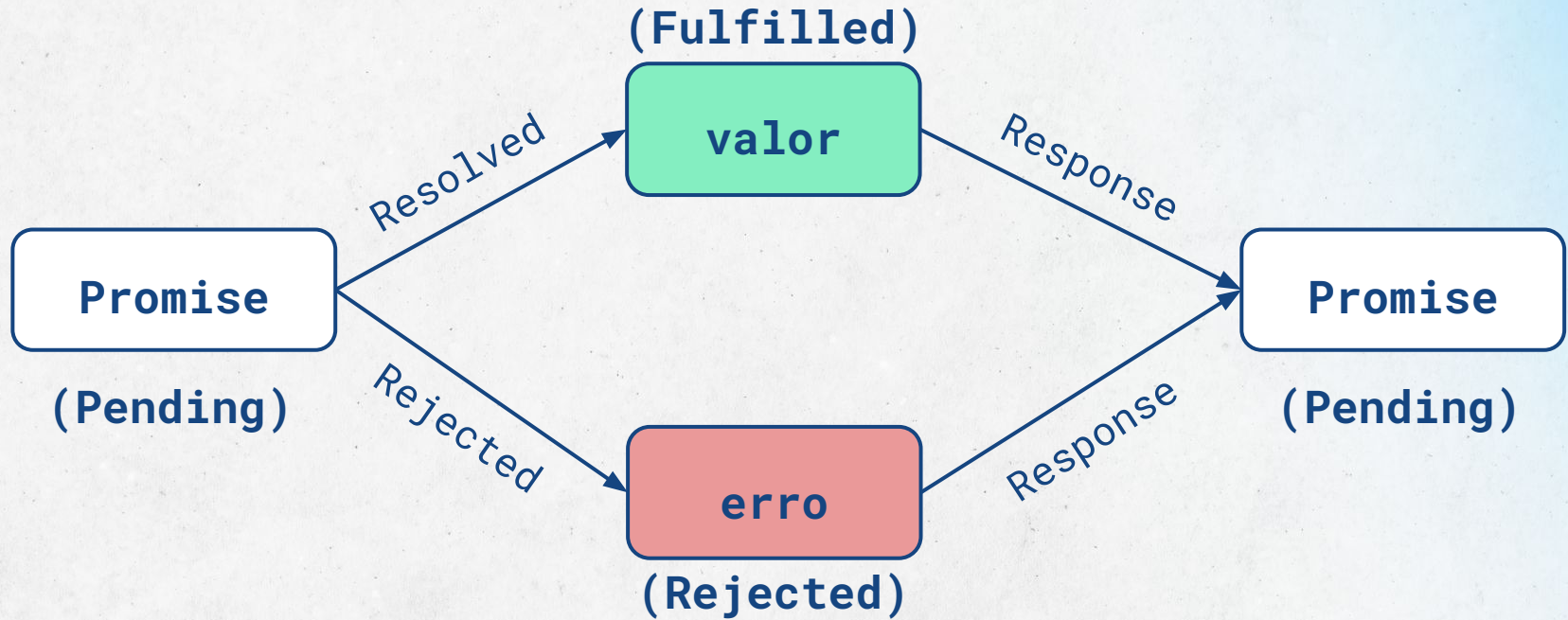
**: Consumindo
e tratando
dados de uma
API**



Métodos das Promises



Promise

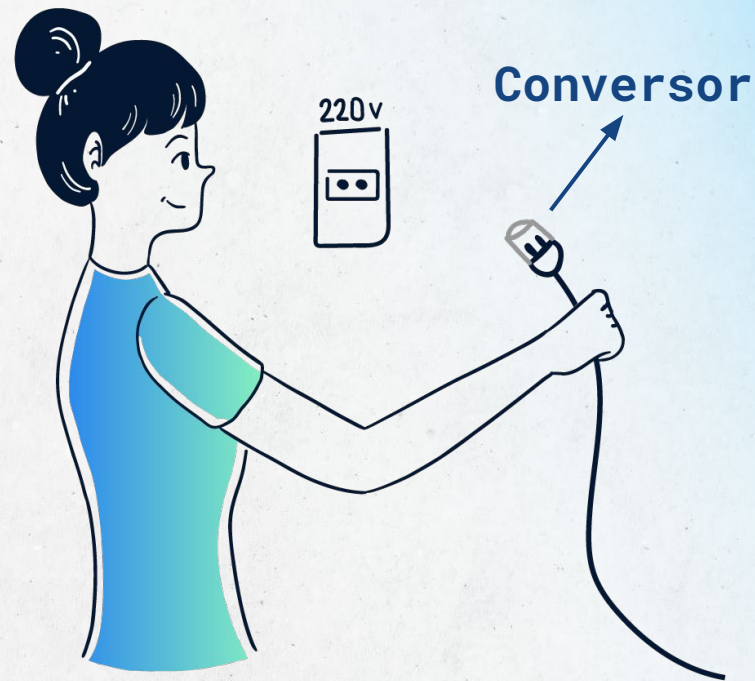


**Você já recebeu uma
encomenda com problema?
Por exemplo da voltagem do
seu produto ser 110v e a da
sua casa ser 220v.**

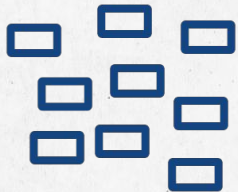


Então para poder usá-lo, foi necessário um conversor.

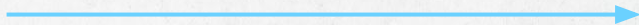
Como seria o conversor do javascript?



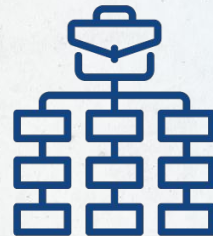
Corpo da
resposta
em bytes



`.json()`



json
formatado



```
1  var consultaCep = fetch('https://viacep.com.br/ws/01001000/json/')
2    .then(response => response.json())
3    .then(result => console.log(result))
4
5  console.log(consultaCep);
```

No fim, o que está escrito aqui é uma sucessão de 'Callbacks' (retornos), sendo:

Utilize a memória de nome **consultaCEP** para buscar (**fetch**) os dados na url '**viaCep**', e então (**then**) armazene o retorno na memória **response**, depois converta os dados da memória **response** em uma arquivo no formato **json**, armazene o retorno na memória **result**. Por fim imprima **result** no console.

Como em cada função eu realizo algo diferente e esta função me retorna (Callback) um dado NOVO, onde o primeiro é o DADO que ele acha no endereço (URL), depois é a conversão deste DADO em JSON, e por último é como este DADO será impresso, eu preciso declarar uma MEMÓRIA para cada retorno.

► Promise {<pending>}

script.js:5

script.js:3

▼ {cep: '01001-000', logradouro: 'Praça da Sé', complemento: 'lado ímpar', u
nidade: '', bairro: 'Sé', ...} ⓘ
 bairro: "Sé"
 cep: "01001-000"
 complemento: "lado ímpar"
 ddd: "11"
 estado: "São Paulo"
 gia: "1004"
 ibge: "3550308"
 localidade: "São Paulo"
 logradouro: "Praça da Sé"
 regioao: "Sudeste"
 siafi: "7107"
 uf: "SP"
 unidade: ""
 ► [[Prototype]]: Object

Aqui nós temos 2 console.log.

- O primeiro é do consultaCep que é justamente a Promise(promessa que algum dado vai chegar)
- E o segundo é o result em formato JSON que a gente pediu.

Tratament o de erros com catch

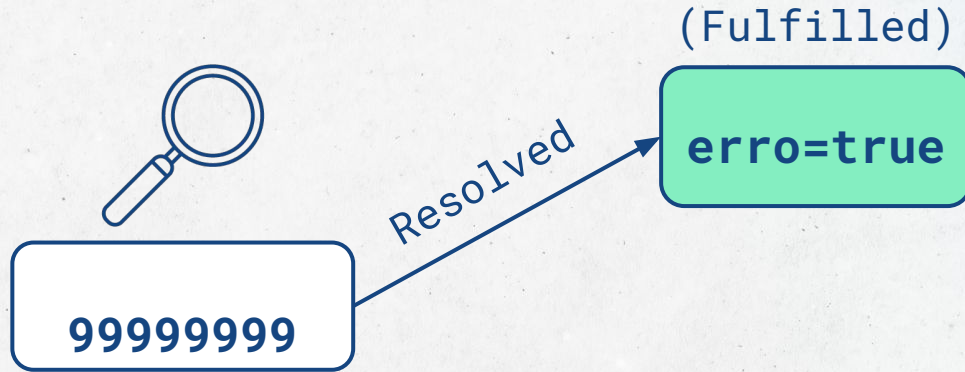




Em um pedido, caso a transportadora não for até sua casa, ela precisa avisar de alguma maneira.







```
var consultaCep = fetch('https://viacep.com.br/ws/0001000/json/')
.then(response => response.json())

//Se o resulta for verdadeiro, ele vai imprimir o resultado no console
.then(result => console.log(result))

//Se o resultado for falso, ele vai imprimir a mensagem de erro no console
.catch(error => console.log('ERRO AO CONSULTAR O CEP:', error));

console.log(consultaCep);
```

► Promise {<pending>} [script.js:6](#)

✖ Access to fetch at 'https://viacep.com.br/ws/0001000/json/' [index.html:1](#) from origin 'http://127.0.0.1:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

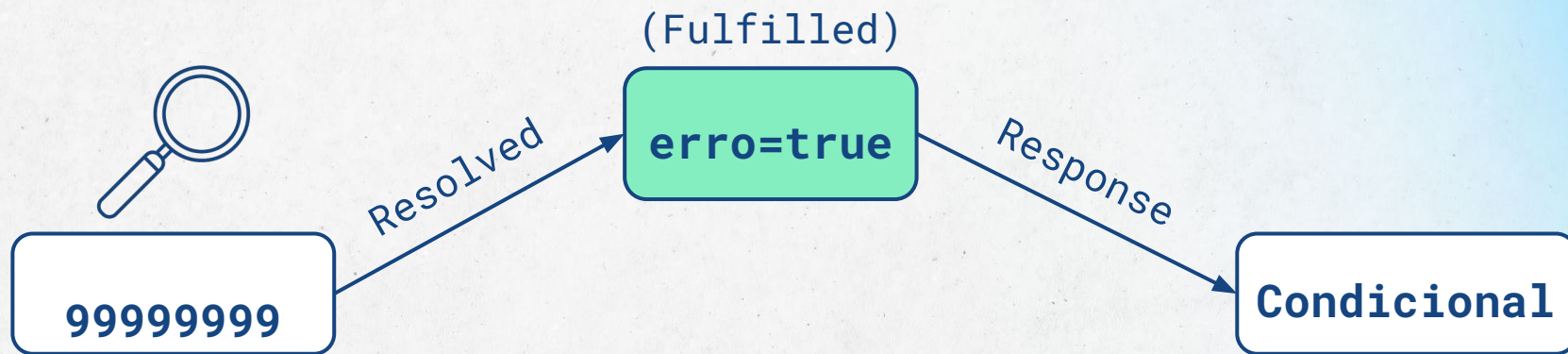
✖ ► GET https://viacep.com.br/ws/0001000/json/ [script.js:1](#) net::ERR_FAILED 400 (Bad Request)

ERRO AO CONSULTAR O CEP: TypeError: Failed to fetch [script.js:4](#)
at [script.js:1:19](#)

[NOVO] Explique os erros do Console usando o Copilot no Edge: clique em  para explicar um erro. [Saiba mais](#)

>

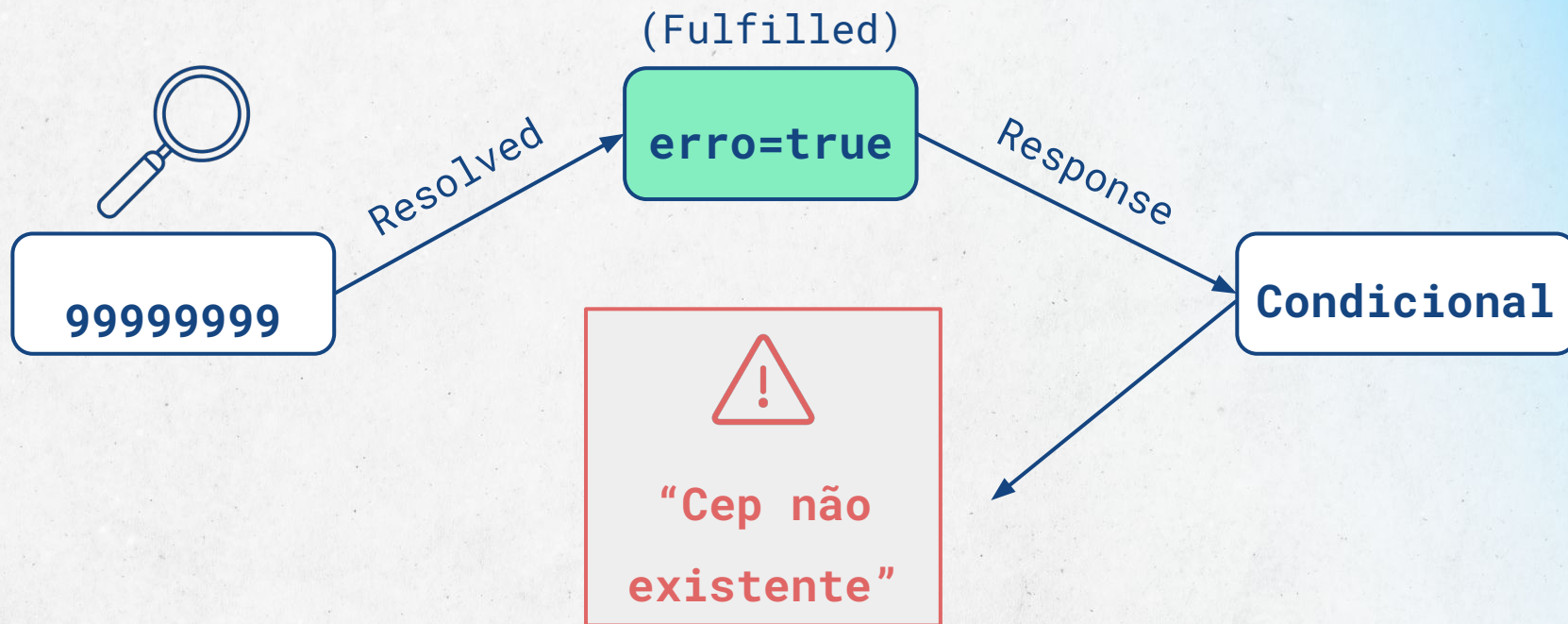
Não mostrar novamente



Validação do CEP

Quando consultado um CEP de formato inválido, exemplo: "950100100" (9 dígitos), "95010A10" (alfanumérico), "95010 10" (espaço), o código de retorno da consulta será um **400** (Bad Request). Antes de acessar o webservice, valide o formato do CEP e certifique-se que o mesmo possua **{8}** dígitos. Exemplo de como validar o formato do CEP em javascript está disponível nos exemplos abaixo.

Quando consultado um CEP de formato válido, porém inexistente, por exemplo: "99999999", o retorno conterá um valor de "erro" igual a "true". Isso significa que o CEP consultado não foi encontrado na base de dados. Veja como tratar este "erro" em javascript nos exemplos abaixo.



Se tiver, ele irá criar um erro com a mensagem
"CEP não existente".

```
1 var consultaCep = fetch('https://viacep.com.br/ws/35185909/json/')
2   .then(response => response.json())
3
4 //Se o resulta for verdadeiro, ele vai imprimir o resultado no console
5   .then(result => {
6     if (result.erro) {
7       throw Error('CEP não encontrado');
8     } else {
9       console.log(result);
10    }
11  })
12
13 //Se o resultado for falso, ele vai imprimir a mensagem de erro no console
14   .catch(error => console.log(error));
15
16 console.log(consultaCep);
```

```
▼ Promise {<pending>} i script.js:16
  ► [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: undefined
ERRO AO CONSULTAR O CEP: Error: CEP não encontrado script.js:14
at script.js:7:11
```

Recapitulando:

Além de só dar 'erro', estamos também informando o usuário a como lidar com este erro, sendo:

- Na primeira situação o CEP está completo MAS errado, então ele cai no 'throw Error' pois segundo a própria documentação, a resposta do servidor é POSITIVA, mas inválida;
- No 'catch', o CEP está INCOMPLETO, e a resposta do servidor é um ERRO, então ele vai ser pego no CATCH.

Finall

y
Alpha 3.3



JS script.js > ...

```
1  var consultaCep = fetch('https://viacep.com.br/ws/35185909/json/')
2  .then(response => response.json())
3
4  //Se o resulta for verdadeiro, ele vai imprimir o resultado no console
5  .then(result => {
6    if (result.erro) {
7      throw Error('CEP não encontrado');
8    } else {
9      console.log(result);
10    }
11  })
12
13  //Se o resultado for falso, ele vai imprimir a mensagem de erro no console
14  .catch(error => console.log(error))
15
16  //Ao fim, ele vai imprimir a mensagem de processamento concluído no console
17  .finally(mensagem => console.log('Processamento concluído!'));
18
19  console.log(consultaCep);
```

► Promise {<pending>}

[script.js:19](#)

Error: CEP não encontrado
at [script.js:7:11](#)

[script.js:14](#)

Processamento concluído!

[script.js:17](#)